

FIRST PART – ROS PUBLISHER, SUBSCRIBER AND CUSTOM MSG

In this exercise, you will develop a basic structure for **a game controlled through ROS nodes**. The objective is to implement the game logic, where a username is requested, an object's movement is controlled within the game, and the results are communicated between nodes.

You need to create five Nodes: `info_user`, `game_node`, `control_node`, `control_node_pygame` and `result_game`

You need to create Three topics: `user_information`, `keyboard_control`, `result_information`

You need to create one custom msg: `user_msg`

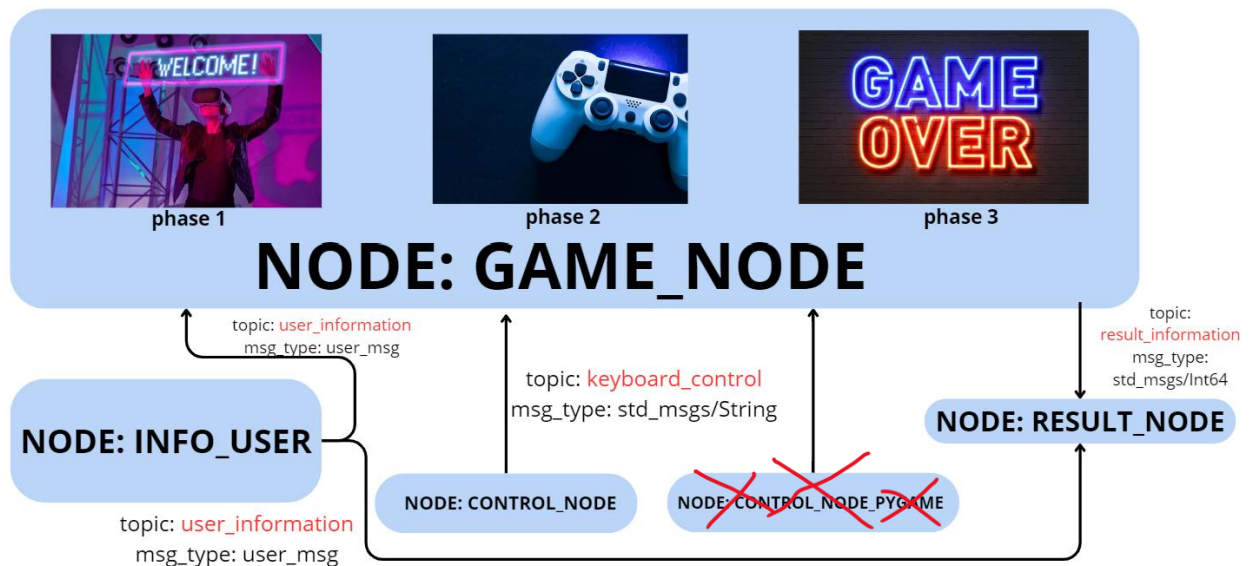
Exercise Description

- **1. Node INFO_USER:**
 - This node should request the player's name, username and the player's age through the terminal.
 - Once the information is obtained, it should send this information to Node **GAME_NODE**. For sending this information we need to use the **topic `user_information`** with the **msg_type: `user_msg`**.
 - CUSTOM MSG – `user_msg`
 - string name
 - string username
 - int64 age
- **2. Node GAME_NODE: (Main Game Node)**
 - This node is responsible for the game logic and contains three phases:
 - **Phase 1: Welcome:** In this initial phase, the node receives the player's information from Node **INFO_USER** and prints in the screen the name of the user.
 - **Phase 2: Game:** In this phase, the user should control the movement of an object using the arrow keys (up, down, left, right).

Node Control_node: this node control the movement of the player.

- Send the movement information to Node **GAME_NODE** with a string. For sending this information we need to use the **topic `keyboard_control`** with the **msg_type: `std_msgs/String`**.
- The Value of the String is:
 - Right movement: "RIGHT"
 - Left movement: "LEFT"
 - Up movement: "UP"
 - Down movement: "DOWN"

- If you need another movement you need to use the same format, all in capital letters
 - The goal of this phase is play the game.
 - **Final:** In this final phase, the node calculates the player's final score and sends this information to Node **RESULT_NODE**. The topic that this node use to communicate the score information is *result_information* with *the msg_type= std_msgs/Int64*
- **5. Node RESULT_NODE:**
 - This node receives the player's information from **Node INFO_USER** with the **topic user_information**.
 - This node receives the player's score from **Node GAME_NODE** with the **topic result_information**
 - It should display a final message with the score and the username.



Technical Requirements

1. Node Structure:

- Each node should be defined within a separate class structure. You need to use different python files.
- No global variables are allowed. All data must be encapsulated within each node's class, ensuring modularity and preventing unintended interactions.

2. Phases Implementation in Node GAME_NODE:

- In Node GAME_NODE (the main game node), implement each game phase (Welcome, Game, and Final) as a separate method within the class.
- Ensure that each method is self-contained, handling the logic for that particular phase and facilitating smooth transitions between phases without relying on global state.

3. Communication Between Nodes:

- Node INFO_USER should publish the player's information as a ROS message to be received by Node GAME_NODE.
- Node GAME_NODE should publish the final score as a ROS message to be received by RESULT_NODE.
- Use appropriate ROS message types (std_msgs/String for the username, the player's name, and std_msgs/Int32 or similar for the score and the player's age) for inter-node communication.
- Node GAME_NODE should act as a publisher for the score, and Node RESULT_NODE should act as a subscriber to receive and display the final score.

4. Keyboard Control Alternatives:

- Implement keyboard control for the game phase. This implementation should be communicated with GAME_NODE

5. Logging and Transition Messages:

- Add log messages to inform about transitions in all the nodes
- Add log messages to inform about transitions between each phase in Node GAME_NODE. For example:
 - "Welcome phase started."
 - "Game phase started."
 - "Final phase reached, calculating score."
- This logging will help with debugging and provide clear feedback to the user and developer about the game's state.

6. Documentation:

- Create a README.md file that explains how to run each node, including any required dependencies such as pygame.
- Include instructions for installing dependencies, running each node, and an overview of the node-to-node communication process.

7. Save the changes

- Upload to the github repository in the **ROS branch**
- **Mandatory to record all changes in GitHub in the project's ROS branch.**